Aim: To implement AJAX applications that enable asynchronous communication with servers without reloading the webpage.

Q1. What is AJAX?

AJAX (Asynchronous JavaScript and XML) is a web development technique that allows web pages to communicate with a server asynchronously—sending and receiving data in the background without reloading the entire webpage. It enables dynamic content updates, improving user experience and performance.

Q2. What is an Asynchronous operation?

An **asynchronous operation** is a task that runs independently of the main program flow, allowing the program to continue executing other code while waiting for the task to complete.

Q3. Implement AJAX applications that enable asynchronous communication with servers without reloading the webpage.

Code (index.html):

```
<!doctype html>
<html lang="en">
    <head>
        <meta charset="UTF-8" />
        <meta
            name="viewport"
            content="width=device-width,
initial-scale=1.0, user-scalable=yes"
        />
                                    </head>
    <body>
        <div class="ajax-card">
            <div class="exp-header">
                <h1>
                    AJAX Application
                    <small>Async
Communication</small>
                </h1>
                <p>
                    Fetch and display
data dynamically without page reload |
                    JSONPlaceholder API
                </p>
            </div>

            <div class="control-panel">
                <div class="btn-group">
                    <button
id="fetchPostsBtn" class="btn btn-
primary">
                        GET Posts (AJAX)
                    </button>
                    <button id="clearBtn"
class="btn btn-outline">
                        Clear Data
                    </button>
                </div>
                <div class="status-area"
id="statusWidget">
                    <span class="status-
badge" id="statusLed"></span>
                    <span
id="statusText">Ready</span>
                </div>
            </div>

            <div class="output-
container">
                <div class="section-
title">Live Data from Server</div>
                <div id="dataDisplay"
class="data-grid">
                    <div
class="placeholder-message">
                        Click the
<strong>"GET Posts"</strong> button to
fetch
                        data
asynchronously.<br />
                        AJAX request will
be sent to JSONPlaceholder API and
                        display posts
dynamically.
                    </div>
```

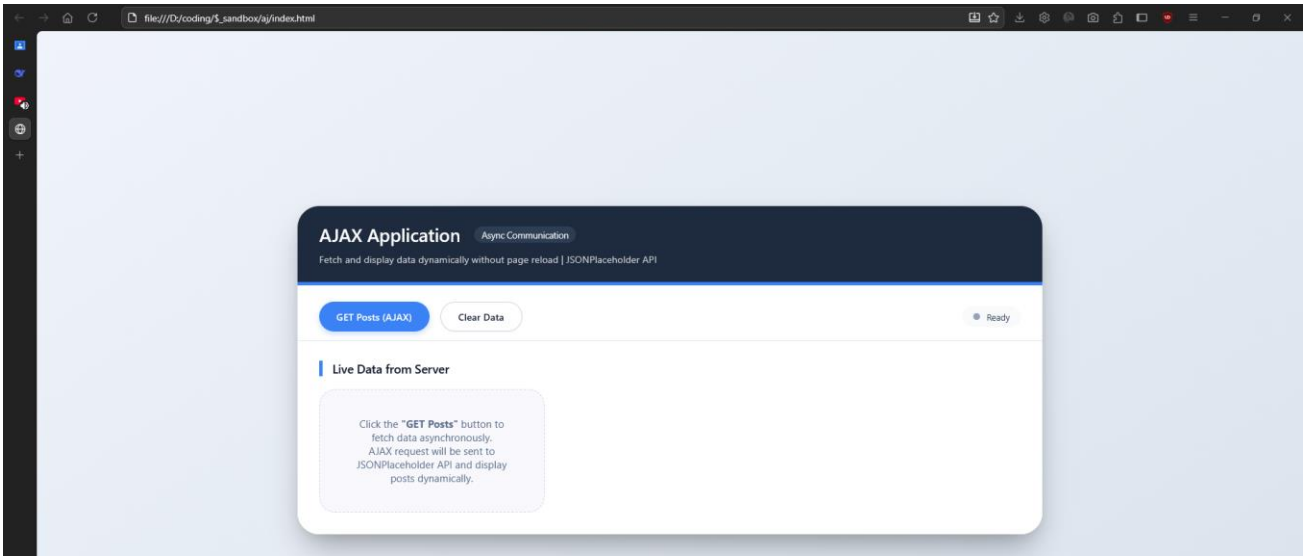
```
            </div>
        </div>
    </div>

    <script>
        const fetchBtn =
document.getElementById("fetchPostsBtn");
        const clearBtn =
document.getElementById("clearBtn");
        const dataDisplayDiv =
document.getElementById("dataDisplay");
        const statusLed =
document.getElementById("statusLed");
        const statusTextSpan =
document.getElementById("statusText");

        function
updateStatus(message, isLoading = false,
isError = false) {
            statusTextSpan.textConten
t = message;
            if (isLoading) {
                statusLed.className =
"status-badge active";

                statusLed.style.backg
round = "#facc15";
                statusLed.style.boxSh
adow = "0 0 0 2px #fef9c3";
            } else if (isError) {
                statusLed.className =
"status-badge";
                statusLed.style.backg
round = "#ef4444";
                statusLed.style.boxSh
adow = "0 0 0 2px #fee2e2";
            } else {
                statusLed.className =
"status-badge";
                statusLed.style.backg
round = "#22c55e";
                statusLed.style.boxSh
adow = "none";
            }
            if (!isLoading &&
!isError) {
                statusLed.style.backg
round = "#22c55e";
            }
            if (!isLoading &&
!isError && message === "Ready") {
                statusLed.style.backg
round = "#94a3b8";
            }
        }

        function
showLoadingSkeleton() {
            dataDisplayDiv.innerHTML
= `
        <div class="placeholder-
message" style="display: flex; align-
items: center; justify-content: center;
gap: 12px;">
            <span
class="loader"></span> Fetching data from
server, please wait...
        </div>
    `;
        }

        function
renderPosts(postsArray) {
            if (!postsArray ||
postsArray.length === 0) {
                dataDisplayDiv.innerH
TML = `<div class="placeholder-message">
No posts found. Try again later.</div>`;
                return;
            }

            const fragment =
document.createDocumentFragment();
            const gridContainer =
document.createElement("div");
            gridContainer.className =
"data-grid";

            const limitedPosts =
postsArray.slice(0, 12);

            limitedPosts.forEach((pos
t) => {
                const postCard =
document.createElement("div");
                postCard.className =
"post-card";

                const titleText =
post.title ? post.title : "Untitled";
                const bodyText =
post.body
                    ? post.body
                    : "No content
available.";

                postCard.innerHTML =
`
            <div class="post-title">
${escapeHtml(titleText.substring(0,
70))}${titleText.length > 70 ? "..." :
""}</div>
```

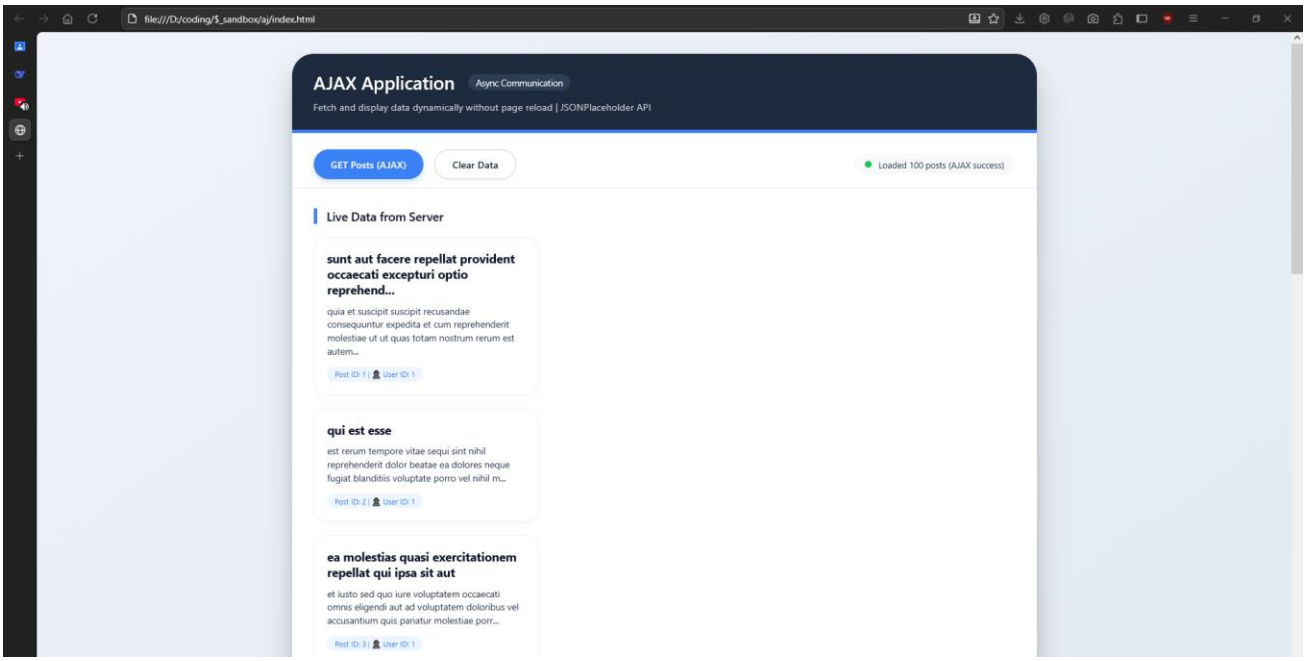
```
                <div class="post-
body">${escapeHtml(bodyText.substring(0,
130))}${bodyText.length > 130 ? "..." :
""}</div>
                <div class="post-meta">
Post ID: ${post.id} | User ID:
${post.userId}</div>
            `;
                    gridContainer.appendC
hild(postCard);
                });

                fragment.appendChild(grid
Container);

                dataDisplayDiv.innerHTML
= "";
                dataDisplayDiv.appendChil
d(fragment);
            }

            function escapeHtml(str) {
                if (!str) return "";
                return str
                    .replace(/[&<>]/g,
function (m) {
                        if (m === "&")
return "&amp;";
                        if (m === "<")
return "&lt;";
                        if (m === ">")
return "&gt;";
                        return m;
                    })
                    .replace(/[\uD800-
\uDBFF][\uDC00-\uDFFF]/g, function (c) {
                        return c;
                    });
            }

            function showError(message) {
                dataDisplayDiv.innerHTML
= `
            <div class="error-message">
               ${escapeHtml(message)}<br>
                <span style="font-
size:0.8rem;"> Tip: Check your internet
or try again later.</span>
            </div>
        `;
                updateStatus(`Error:
${message.substring(0, 50)}`, false,
true);

                setTimeout(() => {
                    if
(statusTextSpan.innerText.includes("Error
")) {
                        if
(!window.activeAjaxRequest) {
                            updateStatus(
"Ready", false, false);
                        }
                    }
                }, 3000);
            }

            function fetchPostsWithAJAX()
{
                updateStatus("Fetching
data...", true, false);
                showLoadingSkeleton();

                const xhr = new
XMLHttpRequest();

                const apiUrl =
"https://jsonplaceholder.typicode.com/pos
ts";

                xhr.open("GET", apiUrl,
true);

                xhr.setRequestHeader("Acc
ept", "application/json");

                xhr.onreadystatechange =
function () {
                    if (xhr.readyState
=== 4) {
                        if (xhr.status
=== 200) {
                            try {
                                const
responseData = JSON.parse(
                                    xhr.r
esponseText,
                                );

                                if (
                                    Array
.isArray(responseData) &&
                                    respo
nseData.length > 0
                                ) {
                                    rende
rPosts(responseData);
                                    updat
eStatus(
```

```
                                        `
Loaded ${responseData.length} posts (AJAX
success)`,
                                        f
alse,
                                        f
alse,
                                    );

                                    setTi
meout(() => {
                                        i
f (

  statusTextSpan.innerText.includes(

      "Loaded",

  )
                                        )
{

  updateStatus("Ready", false, false);
                                        }
                                    },
2500);
                                } else {
                                    dataD
isplayDiv.innerHTML = `<div
class="placeholder-message"> Server
returned empty data. Nothing to
display.</div>`;
                                    updat
eStatus(
                                        "
No data received",
                                        f
alse,
                                        f
alse,
                                    );
                                }
                            } catch
(parseError) {
                                showError
("Invalid JSON response from server.");
                            }
                        } else {
                            let errorMsg
= `HTTP error ${xhr.status}:
${xhr.statusText}`;
                            if
(xhr.status === 0) {
                                errorMsg
=
                                    "Netw
ork error or CORS / request blocked.";
                                }
                                showError(err
orMsg);
                            }

                            window.activeAjax
Request = false;
                        }
                    };

                    xhr.onerror = function ()
{
                        showError(
                            "Network failure.
Please check your internet connection.",
                        );
                        window.activeAjaxRequ
est = false;
                    };

                    xhr.timeout = 15000;
                    xhr.ontimeout = function
() {
                        showError(
                            "Request timeout.
Server took too long to respond.",
                        );
                        window.activeAjaxRequ
est = false;
                    };

                    window.activeAjaxRequest
= true;

                    xhr.send();
                }

                function clearData() {
                    dataDisplayDiv.innerHTML
= `
                <div class="placeholder-
message">
                     Data cleared. Click
<strong>"GET Posts"</strong> to fetch
fresh data via AJAX.
                </div>
            `;
                    updateStatus("Ready",
false, false);
                }

                fetchBtn.addEventListener("cl
ick", () => {
                    fetchPostsWithAJAX();
```

```
            });

            clearBtn.addEventListener("cl
ick", () => {
                clearData();
            });
                updateStatus("Ready", false,
false);

        </script>
    </body>
</html>
```

Output:





Conclusion:

Successfully implemented AJAX applications that enable asynchronous communication with servers without reloading the webpage.